PANDAS

↳ Use case of pandas

- Data cleaning
- Data analysis
- Data Transformation
- Data visualization (Basic Level)
- Data Aggregration
- File Handling
- Data Filtering & selection
- Time series Analysis

Import pandas as pd

↳ Topics to cover

1. Series
2. Data Frames
3. Missing Data
4. Merging. joining concatination
5. GroupBy & Aggregations
6. Pivot Tables & crosstab
7. Operations
8. Working with real data (Finding Data)

PROJECTS

9. Feature Extraction Project
10. Data capstone Project

1. PANDAS SERIES

→ **Pandas series / Series** - It is a 1-D labeled array capable of holding any data type (unlike standard arrays which only hold single data types)

* This Axis label are collectively called as "index".

ex -

```
import pandas as pd
data = [10, 20, 30, 40]
S = pd.series(data)
print(s)
```

*if we don't provide index value it get autogenerated. starting from 0 to end.

output =

index	Series
0	10
1	20
2	30
3	40

dtype : int64

* Data in a series is stacked vertically, and the index numbering (0, 1, 2) acts as "labels".

→ **Creating series**

code = `Pd.Series(my_list)`

#We are using Jupyter Notebook

[1]:

ex code

```
import numpy as np
import pandas as pd
labels = ['a', 'b', 'c']
my_list = [10, 20, 30]
arr = np.array([10, 20, 30])
d = {'a': 10, 'b': 20, 'c' 30}
```

[2]: `Pd.Series(my_list)` # with creating series from python list

output:
```
0   10
1   20
2   30
dtype: int64
```

Use print in compiler
↳ print (Pd.Series(my_list))

[3]: `Pd.Series(my_list, index=labels)` # creating series with index=labels

output:
```
a   10
b   20
c   30
dtype: int64
```

[4]: `Pd.Series(arr)` # creating series from array

output:
```
0   10
1   20
2   30
dtype: int64
```

[5]: `Pd.Series(d)` # creating series from dictionary

output:
```
a   10
b   20
c   30
dtype: int64
```

→ **Series Vs Dataframes**

Series	Dataframes
It is a single column of data. & It is the foundational building block.	Combining Multiple series creates a DataFrame which is a table used in data analysis.

2. DATAFRAMES

→ **DataFrames-**

A Dataframe is a multidimensional data structure (like a Table) created by combining Multiple Pandas series/series together side by side.

→ **Creating a DataFrame**

(A) From a Dictionary

Code = `pd.DataFrame(data)`

We are using Jupyter Notebook, use print command like print(pd.DataFrame(data)) in python compiler.

In [1]:

ex code =

```
data ={
        'Name': ['John', 'Anna', 'Peter', 'Linda' ],
        'Age': [28, 34, 29, 42],
        'city': ['New York', 'Paris', 'Berlin', 'London'],
        'salary': [65000, 70000, 62000, 85000]

}
```

In [2]:

```
df = pd.DataFrame(data)
df
```

Use print command in normal compiler like print(df)

Out[2]:

Output =

	Name	Age	city	salary
0	John	28	Newyork	65000
1	Anna	34	Paris	70000
2	Peter	29	Berlin	62000
3	Linda	42	London	85000

B. From a List of Lists:

Code:

```
pd.DataFrame(data_list)
```

If we pass a plain list of lists, Pandas will default to naming the columns with numbers (0, 1, 2, 3)

In[]:

ex code:

```
data_list=[
    ['John',28,'Newyork',65000],
    ['Anna',34,'Paris',70000],
    ['Peter',29,'Berlin',62000],
    ['Linda',42,'London',62000],
]
```

In [2]:

```
df2 = pd.DataFrame(data_list)
columns = ["Name","Age","city","salary"]
df2 = pd.DataFrame(data_list,columns=columns)
df2
```

Out[2]:

Output =

	Name	Age	city	salary
0	John	28	Newyork	65000
1	Anna	34	Paris	70000
2	Peter	29	Berlin	62000
3	Linda	42	London	85000

→ selection & Indexing of columns

In[1]:

```
data_list = [
        ['John' , 28 , 'Newyork' , 65000],
        ['Anna' , 34 , 'Paris' , 70000 ],
        ['Peter' , 29 , 'Berlin' , 62000],
        ['Linda' , 42 , 'London' , 85000] ]
df2 = pd.DataFrame (data_list,
        columns=["Name" ,"Age", "city" ,
                 "salary"])
```

→ we are using this data_list for more further topic.

- **Selecting a column**: code `df2 ["Name"]`

In[2]:

ex code = `df2 [ ["Name" , "city"]]`

Out[2]:

output =

	Name	City
0	John	Newyork
1	Anna	Paris
2	Peter	Berlin
3	Linda	London

• **creating a new column** : Declare a new column name & assign a list of values to it.

Rule:- The number of values provided must perfectly match the no. or rows in the DataFrame.

ex code =

In [3]:

```
df2["Designation"] = ["Doctor", "Eng", "Doctor",
                      "Eng."]
df2
```

output = Out[3]:

	Name	Age	city	salary	Designation
0	John	28	Newyork	65000	Doctor
1	Anna	34	Paris	70000	Eng.
2	Peter	29	Berlin	62000	Doctor
3	Linda	42	London	85000	Eng.

• **Removing columns** code = `df2.drop(0, axis=0)`

(0 = column no., axis=0 → horizontal)

ex code =

In[4]:

```
df2.drop(0, axis=0)
```

output

	Name	Age	city	salary
1	Anna	34	Paris	70000
2	Peter	29	Berlin	62000
3	Linda	42	London	85000

→ Row operations

* **.loc[ ] (Location):** Used to select rows by their "label / index" name. We can pass list to grab Multiple rows at once.

ex code =

In [5]:

```
df2.loc [ [0, 1] ]
```

Output =

Out[5]:

	Name	Age	city	salary
0	John	28	Newyork	65000
1	Anna	34	Paris	70000

* **.iloc [ ] (Index location):** Used to grab rows by their exact integer index position, regardless of what the labels are named.

ex code =

In [1]:

```
data = {
    'Name' : ['John', 'Anna', 'Peter', 'Linda'],
    'Age' : [28, 34, 29, 42],
    'city' : ['NewYork', 'Paris', 'Berlin', 'London'],
    'salary' : [65000, 70000, 62000, 85000]
}
```

In [2]:

```
df = Pd.DataFrame(data)
```

↳ we are using this "data" for further topic

In [3]:

```
df.iloc[3]      # for 3 index or 3column
```

Out[3]:

Output:

```
Name      Linda
Age       42
city      London
salary    85000
Name      :3 , dtype: object
```

→ Selecting subsets (Rows & column Together)

we can chain commands to extract a highly specific subset of our data by combining " `.loc[ ]` " for rows & standard bracket selection for column.

In [4]:

ex code =

```
df.loc[ [0,1] ] [ ["city","salary"] ]
```

[0,1] → column/Index

["city","salary"] → Rows

out[4]:

output:

	city	salary
0	Newyork	65000
1	Paris	70000

→ conditional selection

We can filter a DataFrame to only show row that meet specifice mathematical or logical conditions.

In [1]:

```
data_list= [
    ['John' , 28 , 'NewYork' , 65000],
    ['Anna' , 34 , 'Paris' , 70000] ,
    ['Peter' , 29 , 'Berlin' , 62000],
    ['Linda' , 42 , 'London' , 85000]
]
df2 = pd.DataFrame (data_list)
columns = ["Name", "Age", "city" , "salary"]
df 2 = pd.DataFrame(data_list , columns =columns)
```

- **single condition :**

In [2]:

ex code =

```
df2 [df2 ["Age"] >30]
# If we wan't people whose age is
  above 30
```

output:

Out[2]

	Name	Age	city	salary
1	Anna	34	Paris	70000
3	Linda	42	London	85000

• Multiple conditions

* Use the "&" symbol for AND (both conditions must be true)

```
df[(df[Age]>30) |
(df['salary']>80000)]
```

* Use the "|" symbol for OR (either condition can be true)

→ It's not 1 its | (stick symbol)

ex code :

In [3]:

```
df2[(df2["Age"] > 30) & (df2["city"] == 'Paris')]
```

We only want people whose age is above 30 & their Must be from Paris

output:

Out [3]

	Name	Age	city	salary
1	Anna	34	Paris	70000

3. MISSING DATA

→ Finding Data

In[1]:

```
data = {
    'A' : [1, 2, np.nan, 4, 5],
    'B' : [np.nan, 2, 3, 4, 5],
    'C' : [1, 2, 3, np.nan, np.nan],
    'D' : [1, np.nan, np.nan, np.nan, 5]
}
df = pd.DataFrame(data)
df
```

out[1]:

	A	B	C	D
1	1.0	NaN	1.0	1.0
2	2.0	2.0	2.0	NaN
3	NaN	3.0	3.0	NaN
4	4.0	4.0	NaN	NaN
5	5.0	5.0	NaN	5.0

• **df.isna()** : This command converts the entire DataFrame into a grid of True & False values, where True indicates that a piece of data is missing.

Code :

```
df.isna()
```

ex code =

In[2]

```
df.isna()
```

output =

Out[2]

	A	B	C	D
0	False	True	False	True
1	False	False	False	True
2	True	False	False	True
3	False	False	True	True
4	False	False	True	False

• **df.isna().sum()** :- It Make output, the exact count of missing values per column (eg. column C has 2 missing values, column D has 3)

ex code =

In[3]

```
df.isna().sum()
```

output =

Out[3]

```
A    1
B    1
C    2
D    3
dtype: int64
```

• **df.isna().any()** :- This simply returns a True or False for each column, telling us quickly if a specific column has any missing data at all.

ex code :-

In [4]

```
df.isna().any()
```

Output =

Out [4]

```
A    True
B    True
C    True
D    True
dtype: bool
```

→ Removing Missing Data

- **Standard Drop(df.dropna())**: By Default, It deletes any entire row that contains even a single NaN value.

In [1]

```
data = {
    'A': [1, 2, np.nan, 4, 5],
    'B': [1, 2, 3, 4, 5],
    'C': [1, 2, 3, np.nan, np.nan],
    'D': [1, np.nan, np.nan, np.nan, 5]
}

df = pd.DataFrame(data)
df
```

This "data" is used in further topic

ex code =

In [2]

```
df.dropna()
```

Output:

Out [2]

	A	B	C	D
0	1.0	1	1.0	1.0

like previous example of data that we are using from start, has every row has NaN value. then in output all rows are dropped or deleted.

- **Setting a Threshold (df.dropna(thresh=1))** : It is used to prevent deleting too much data, a threshold is applied.

By setting thresh=1 Pandas is instructed to keep any row that has at least 1 valid (non null) value. this allowed all row to survive the drop because every row contained at least one real number.

ex code =

In [2]

```
df.dropna(thresh=1)
```

output :

Out [2]

	A	B	C	D
0	1.0	1	1.0	1.0
1	2.0	2	2.0	NaN
2	NaN	3	3.0	NaN
3	4.0	4	NaN	NaN
4	5.0	5	NaN	5.0

→ ## Filling the Missing Data

Instead of dropping data, the final section of the Notebook shows three ways to replace the NaN values using .fillna():

- **Filling with a single value**: This blanket operation replaces every single NaN values across the entire DataFrame with a 0.

`df.fillna(0)`

ex code =

```
In [3]
df.fillna(0)
```

output:

```
Out [3]
```

	A	B	C	D
0	1.0	1	1.0	1.0
1	2.0	2	2.0	0.0
2	0.0	3	3.0	0.0
3	4.0	4	0.0	0.0
4	5.0	5	0.0	5.0

- **Targeted Filling with a Dictionary**: We can define a dictionary

`df.fillna(value=values)`

(e.g., values = {'A': 0, 'B': 100, 'C': 300, 'D': 400}) to apply distinct fill rules. The output shows that missing values in column C, were specifically replaced with 300, while missing values in column D were replaced with 400.

ex code =

In [4]:
```
values = {'A' : 0, 'B': 100, 'C' :300, 'D': 400}
df.fillna(value=values)
```

output:

Out [4]

	A	B	C	D
0	1.0	1	1.0	1.0
1	2.0	2	2.0	400.0
2	0.0	3	3.0	400.0
3	4.0	4	300.0	400.0
4	5.0	5	300.0	5.0

- **Statistical Filling**

`df.fillna(df.mean())` : this replaces NaN values with the Mathematical average of that specific column. The output shows the NaNs being replaced by floating-point average.

ex code =

In [5]

```
df.fillna(df.mean())
```

output:

Out [5]

	A	B	C	D
0	1.0	1	1.0	1.0
1	2.0	2	2.0	3.0
2	3.0	3	3.0	3.0
3	4.0	4	2.0	3.0
4	5.0	5	2.0	5.0

fill.na → column wise

drop.na → Row wise

Missing data

works on row wise

Null value are Red flag

4. Merging, Joining & Concatenation

→ Merging DataFrame (Pd.merge)

It is used when we want to combine two DataFrames based on a common column (like same employee_id)

syntax =

```
pd.merge(df1, df2, on='common_column',
         how='style')
```

↳ inner or outer

In[1]:

```
employees = pd.DataFrame({
        'employee_id': [1, 2, 3, 4, 5],
        'name': ['John', 'Anna', 'Peter', 'Linda', 'Bob'],
})
        # this DataFrame 1: Employee information
# this DataFrame 2: salaries information
salaries = pd.DataFrame({
    'employee_id': [1, 2, 3, 6, 7],
    'salary': [60000, 80000, 65000, 70000, 90000],
    'bonus': [5000, 10000, 7000, 8000, 12000]
})
```

In[2]

ex code =

```
pd.merge(employees, salaries, on='employees_id',
         how='inner')
```

Output =

Out [2]:

	employee_id	name	department	salary	bonus
0	1	John	HR	60000	5000
1	2	Anna	IT	80000	10000
2	3	Peter	Finance	65000	7000

↳ **The "on" parameter**: It specifies the column name(s) used to match and combine rows between two DataFrames during a merge.

ex-

```
pd.merge(employees, salaries, on='employee_id', how='inner')
```

↳ **The "how" parameter (Merge styles):**

- **Inner Merge (how='inner')**: It is the default style; it acts as an intersection, only keeping records where the "employee_id" exists in both DataFrames.

ex =

```
pd.merge(employees, salaries, on='employee_id', how='inner')
```

- **Outer Merge (how='outer')**: This keeps all records from both DataFrames (like id's 1 through 4). Because some employees don't have salary data & some salaries don't have

employee data, Pandas fills those missing gaps with NaN.

Ex code =

In [3]

```
Pd.merge(employees, salaries, on='employee_id',
how='outer')
```

output=

Out [3]

	employee_id	name	department	salary	bonus
0	1	John	HR	60000.0	5000.0
1	2	Anna	IT	80000.0	10000.0
2	3	Peter	Finance	65000.0	7000.0
3	4	Linda	IT	NaN	NaN
4	5	Bob	HR	NaN	NaN
5	6	NaN	NaN	70000.0	8000.0
6	7	NaN	NaN	90000.0	12000.0

- **Left Merge (how='left')**: Prioritizes the first (left) DataFrame we provided. It Keeps all records from employees (Id's 1, 2, 3, 4, 5) & only pulls in salary data if it finds a match.

Ex code =

In [4]

```
pd.merge(employees, salaries, on='employee_id',
         how='left')
```

We Mosty Use "inner Merge" because it doesn't give Null values

Out[4]

Output =

	employee_id	name	department	salary	bonus
0	1	John	HR	60000.0	5000.0
1	2	Anna	IT	80000.0	10000.0
2	3	Peter	Finance	65000.0	7000.0
3	4	Linda	IT	NaN	NaN
4	5	Bob	HR	NaN	NaN

- **Right Merge (how='right')**: It prioritizes the second (right) DataFrame. It keeps all records from salaries (ID's 1, 2, 3, 6, 7) & only pulls in department data if a match exists.

In[5]

ex code =

```
pd.merge(employees, salaries, on='employee_id',
                    how='right')
```

Out[5]

Output:

	employee_id	name	department	salary	bonus
0	1	John	HR	60000	5000
1	2	Anna	IT	80000	10000
2	3	Peter	Finance	65000	7000
3	6	NaN	NaN	70000	8000
4	7	NaN	NaN	90000	12000

→ concatenation (pd.concat)

It is used when we won't to "glue" or stack DataFrames together, rather than looking for a common column to merge them on.

In[1]:

```
df1 = pd.DataFrame({
    'A': ['A0', 'A1', 'A2'],
    'B': ['B0', 'B1', 'B2'],
    'C': ['C0', 'C1', 'C2']
})

df2 = pd.DataFrame({
    'A': ['A3', 'A4', 'A5'],
    'B': ['B3', 'B4', 'B5'],
    'C': ['C3', 'C4', 'C5']
})
```

always be on list

In[2]

ex code =

```
pd.concat([df2, df1])
```

By Default, pandas stack DataFrames vertically, placing one on top of the other. The resulting DataFrame will keep it original index numbering (eg row 0, 1, 2, 0, 1, 2

Out[2]

output =

	A	B	C
0	A3	B3	C3
1	A4	B4	C4
2	A5	B5	C5
0	A0	B0	C0
1	A1	B1	C1
2	A2	B2	C2

Horizontal Concatenation (columns / axis = 1)

By providing the axis = 1 parameter, pandas stack the DataFrames side by side.

ex code =

In [3]:

```
pd.concat([df1, df2], axis=1)
```

output =

Out[3]

	A	B	C	A	B	C
0	A0	B0	C0	A3	B3	C3
1	A1	B1	C1	A4	B4	C4
2	A2	B2	C2	A5	B5	C5

→ Joining DataFrames (.join)

It connects DataFrames using their index (row labels) instead of column values.

In [1]:

```
df1 = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie']
}, index=[1, 2, 3])
# this is First DataFrame

# this is second DataFrame
df2 = pd.DataFrame({
    'score': [85, 90, 75]
}, index=[2, 3, 4])
```

- **Outer Join**

`df1.join(df2, how='outer')` : an outer join will keep all unique indices from both DataFrames (1, 2, 3, 4) and fill any missing data with NaN.

ex code =

In [2]:
```
df1.join(df2, how='outer')
```

output:

Out [2]

	name	score
1	Alice	NaN
2	Bob	85.0
3	charlie	90.0
4	NaN	95.0

- **Standard Join**

`df2.join(df1)` : If we don't specify the how parameter, it defaults to a Left join. Because df2 was called first, it acts as the left DataFrame & keep its indices (2, 3, 4), pulling in the corresponding names from df1, where possible & leaving index 4 as NaN since it doesn't exist in df1.

ex code =

In [3]
```
df2.join(df1)
```

output =

Out [3]

	score	name
2	85	Bob
3	90	charlie
4	75	NaN

5. Group & Aggregation

↳ **The dataset contains two types of data:**

- categorical data :- Label or groups like category A/B & store S1/S2.
- Numeric data :- Measurable values like sales & quantity

✱ **Easy Understanding:**

- category/store → used for grouping
- sales/quantity Analysis → used for calculation & analysis.

→ **Group By operations**

It is primarily used to group data based on categorical/qualitative variable.

↳ They represent qualities or categories instead of numeric values.

ex- • Gender = Male/Female ; store : S1/S2
• category = A/B etc.

- **The Hidden Objects:** When we simply run "df.groupby('category')" Pandas does not output a readable table Instead, it creates a "GroupBy object", in memory.

To actually see results, we must target a quantitative column (like 'sales') & apply a mathematical function (like .sum())

A column that stores numbers that represent quantity (amount, size, count etc.)

• Grouping by a single column:

In the grouping, it groups DataFrame by the unique values in 'category' (A & B) & calculate the total sum of sales for each.

Code:

```
df.groupby('category')['sales'].sum()
```

In [1]:

```
data = {
    'category': ['A','B','A','B','A','B','A','B'],
    'store' : ['S1','S1','S2','S2','S1','S2','S2','S1'],
    'sales' : [100,200,150,250,120,180,200,300],
    'Quantity': [10,15,12,18,8,20,15,25],
    'Date':pd.date_range('2023-01-01',periods=8)
}
df = pd.DataFrame(data)
```

In [2]:

ex code =

```
cat = df.groupby('category')['sales'].sum()
cat
# Group by category & calculate the sum of sales
```

cat stores result for future use

Output = Out[2]

```
category
A    590
B    930
Name: sales, dtype: int64
```

* We can easily swap the target;
for instance,

code = `df.groupby('store')['sales'].sum()` , groups the data by S1 & S2.

ex code = In.[3]

```
cat = df.groupby('store')['sales'].sum()
cat
# Group by store & calculate the sum of sales
```

Output = Out[3]

```
store
S1    920
S2    780
Name: sales, dtype: int64
```

• Grouping by Multiple columns:

Used to group more than one category, we must pass the column names inside "list[]".

In[4]

ex code =

```
cat = df.groupby(['category','store'])['sales'].sum()
cat
#group by Multiple colum
#group by category & store
```

output =

out[4]

category	store	
A	S1	220
	S2	350
B	S1	500
	S2	430

Name: sales, dtype: int64

→ Aggregation Methods / Aggregation

"Aggregation" refers to combining multiple data points & applying a statistical math function to summarize them into a single result.

• Basic Aggregation Functions: We can apply individual math operations directly to a column.

Common operations include:

* .mean() - Average
* .median() - Middle value
* .min() - Lowest value
* .max() - Highest value
* .count() - Total no of non-null entries
* .std() - standard deviation → How much values vary from the mean

ex code =

In [5]

```
df ['sales'].mean()
#Mean median min max count std
```

output =

Out [5]

```
187.5
```

→ The ".agg()" Function:

Instead of running these statistical operations one by one, we can use the ".agg" functions to execute multiple operation simultaneously.

ex code =

In [6]

```
df['sales'].agg(['sum','mean','min','max','count'
                 ,'std', 'median'])
```

output =

Out [6]

```
sum       1500.000000
mean       187.500000
min        100.000000
max        300.000000
count        8.000000
std         66.062741
median     190.000000
Name: sales, dtype: float64
```

6. Pivot Tables & Crosstab

group by creates objects not DataFrames that why we can't see object on o/p.

that why we use agg funct like .sum(), .mean() etc

6. Pivot Tables & Crosstab

→ Pivot Tables (pd.pivot_table)

It is used to reshape our Dataframe. Instead of looking at raw list of index number.

We can specify categorical columns to act as our new rows (index) & columns, creating a custom summary table.

Syntax:- `pd.pivot_table()`. We need must add or provide the values, index & columns parameter.

In[1]

```
data = {
    'Date' : pd.date_range('2023-01-01', periods=20),
    'Product' : ['A', 'B', 'C', 'D'] *5,
    'Region' : ['East','West','North','south','East','West',
                'North','south','East','West','North',
                'south','East','West','North','south',
                'East','West','North','south'],
    'sales' : np.random.randint(100,1000,20),
    'Units' : np.random.randint(10,100,20),
    'Rep' : ['John','Mary','Bob','Alice','John','Mary',
             'bob','Alice','John','Mary','Bob','Alice',
             'John','Mary','Bob','Alice','John','Mary',
             'Bob','Alice']
}
```

In [2]

```
df = pd.DataFrame(data)
df['Month'] = df['Date'].dt.month_name()
df['Quater'] = 'Q' + df['Date'].dt.quater.astype(str)
df
```

In [3]

ex code =

```
pd.pivot_table(df, values = "sales", index = 'Region',
               columns = "Products")
```

output =

Out [3]

Product	A	B	C	D
Region				
East	474.2	NaN	NaN	NaN
North	NaN	NaN	619.0	NaN
South	NaN	NaN	NaN	485.0
West	NaN	527.2	NaN	NaN

How it works : This group the data so that the distint Regions (East, North, South, West) become the row labels, the Products (A, B, C, D) become the column headers, & it populates the table using the data from the sales column.

* Hadling Missing combinations (NaN) : If a specific combination does not exist in your dataset (for example, if Product 'A' was never sold in the 'North region'), Pandas will fill ~~that~~ intersection in the pivot table with NaN (Not a number).

→ Aggeration Rules in Pivot tables

- Default Math (fun fact) :- When a pivot table groups multiple rows together, it defaults to calculating the mean (average) of those values.

- Changing the Math (aggfunc): We can change this default mathmatical operation by adding the aggfunc parameter.

ex code =

In[4]

```
pd.pivot_table(df, values="sales", index='Region',
columns="Product", aggfunc='median')
```

output =

Out[4]

Product	A	B	C	D
Region				
East	415.0	NaN	NaN	NaN
North	NaN	NaN	596.0	NaN
South	NaN	NaN	NaN	626.0
West	NaN	535.0	NaN	NaN

- Numeric Rule :- The column we assign to values must contain numeric data (like sales or units). If you try to pass a text-based/categorical column (like 'Quater') into values,

Pandas will throw an error because it cannot calculate mathematical average or medians on strings.

→ Analyzing Multiple values

We can pass a list to the values parameter to analyze multiple datasets simultaneously.

ex cod =

In[5]

```
pd.pivot_table(df, value=['sales', 'units'],
index='Region', columns='Product')
```

output = out[5]

	sales							
Product	A	B	C	D	A	B	C	D
Region								
East	474.2	NaN	NaN	NaN	38.6	NaN	NaN	NaN
North	NaN	NaN	617.0	NaN	NaN	NaN	42.0	NaN
South	NaN	NaN	NaN	495.0	NaN	NaN	NaN	64.4
West	NaN	529.2	NaN	NaN	NaN	63.6	NaN	NaN

It generate a wider table that shows the mathematical summaries for both & Units broken down by Region & product

→ Cross Tabs (pd.crosstab)

A crossTab works almost identically to a Pivot table, but they have one crucial difference: Cross Tabs are explicity used for counting occurances.

ex code =

In[6]

```
pd.crosstab(df['Region'], df['Product'])
```

This does not add up the sales; instead, it look at the entire dataset & simply counts the frequency of a category. The output tells us, for

output =

Product	A	B	C	D
Region				
East	5	0	0	0
North	0	0	5	0
South	0	0	0	5
West	0	5	0	0

7. OPERATIONS

• **Data Preprocessing** : Before we analyze data or feed it into machine learning algorithms, we must understand its structure & prepare it.

These basic operations are the foundational steps of "Data Preprocessing".

→ **DataFrames Basic Operations**.

In[1]:

```
df1 = pd.DataFrame({
    'A' :.[1,2,3,4,5],
    'B' : [10, 20, 30, 40, 50],
    'C' : [100, 200, 300, 400, 500]
})
```

1.) **df.shape** : This attribute returns a tuple representing the dimension of our Data Frame (rows, column)

ex code =

In[2]

```
df1.shape
```

output:

Out[2]

```
(5,3)
```

2.) **df.columns** : It returns a list of all column names (headers) in our DataFrame, which is useful when dealing with datasets that have dozens of columns.

ex code =

```
In[3]:
df1.columns
```

Output =

```
Out[3]:
Index(['A','B','C'],dtype='object')
```

3) df.info() :- It is a crucial method that provides a summary of the DataFrame. It tells us the total no. of non-null entries in each column & the specific data type. It is extremely useful for quickly spotting if any column contain missing values.

ex code =

```
In[4]:
df1.info()
```

Output =

```
Out[4]:
<class 'panda.core.frame.DataFrame'>
RangeIndex : 5 entries , 0 to 4
Data columns (total 3 columns)
#  column    Non-Null count    Dtype
0  A         5 non-null        int64
I  B         5 non-null        int64
2  C         5 non-null        int64
dtypes : int64(3)
memory usage : 252.0 bytes
```

4.) df1.describe() :- It instantly generates descriptive statistics for all numeric columns in our DataFrame.

In [5]:

ex code =

```
df1.describe()
```

output =

Out [5]:

count	5.000000	5.000000	5.000000
mean	3.000000	30.000000	300.000000
std	1.581139	15.81138	158.113883
min	1.000000	10.000000	100.000000
25%	2.000000	20.000000	200.000000
50%	3.000000	30.000000	300.000000
75%	4.000000	40.000000	400.000000
max	5.000000	50.000000	500.000000

→ **Broadcasting (Direct Mathematical operations)**

We can directly apply simple mathematical operations to an entire column at once.

- **How it works** :- This command iterates through every single value in column 'A' & adds 10 to it instantly (eg. 1 becomes 11, 2 becomes 12 etc.)

In [6]:

ex code =

```
df1['A'] + 10
```

Output =

Out[6]:

```
0    11
1    12
2    13
3    14
4    15
Name: A, dtype: int64
```

→ **Applying custom functions**

.apply() : While broadcasting works for basic math, something you need to run complex, customized python logic on our data. This is where the ".apply()" method is used.

- **Method A: Using standard Python Functions**

We can define a standard python function, such as one to calculate squares:

Then, target a specific column & pass our function's name inside the apply method:

Pandas will take every value in column 'B' and process it through our custom function.

In[7]

ex code =

```
def square(x):
    return x**2
df1["B"].apply(square)
```

Output:

Out [7]

0	100
1	400
2	900
3	1600
4	2500

Name: B, dtype: int64

- **Method B: Using Lambda Expressions**

Instead of writing a full block of function code, we can use inline Lambda functions for quick operations.

How it work: It does the exact same thing as the custom function above, squaring every value in column 'B'.

ex code:

In [8]

```
df1['D'] = df1["B"].apply(lambda x: x**2)
```

Output:

Out[8]

	A	B	C	D
0	1	10	100	100
1	2	20	200	400
2	3	30	300	900
3	4	40	400	1600
4	5	50	500	2500

8. Finding Data

→ **Why Real data Needed** :-

Earlier → Data was manually created

- Python lists
- Dictionaries
- Numpy arrays

↳ Read-world ML → Uses large external datasets.

- Not typed Manually
- Imported from files or databases

→ **Pandas versatality**

Pandas can read data from multiple formats:

- CSV
- Excel
- JSON
- Text/TSV
- HTML tables
- SQL databases
- Parquet / Feather
- Pickle/ HDF5 / ORC
- Google Sheet

Pandas = Universal data loader

→ Where to find Real data

↳ **"kaggle"**

- Huge collection of free dasets
- covers:
  - * Health(heart attack data)
  - * Music (spotify stats)
  - * Agriculture
  - * Anime dataset.
- Also provides:
  - * online coding notebooks
  - * ML competitions

→ Loading Data into Pandas

- **The Function:** we use the `pd.read_csv()` function to load the file into our environment.

- **Providing Path:** Inside the function, we must pass the exact location (path) of our file wrapped in quotes.

ex - `pd.read_csv("anime.csv")`

- **Different Folder:** If it is stored elsewhere, we must provide the full system path

ex: `df = pd.read_csv("C:/Users/Downloads/anime.csv")`

• **The 'r' string Trick:** When typing out file paths in python, the slashes can sometimes trigger "escape sequence" errors.

To prevent this, it is a best practice to put lower case `r` (which stands for "raw string") right before the opening quote of our path string.

ex - `df = pd.read_csv(r"c:\Users\Downloads\anime.csv")`

→ Initial Data Inspection

Once we load a massive dataset (like one with thousands of rows), we shouldn't print the whole thing.

• **df.head()** : instead, we use this `df.head` .

ex = `df.head`

This function gives us a quick visual snapshot of our newly loaded DataFrame by displaying only the top 5 rows, allowing us to inspect the column names & Data structure before we begin our analysis

Project 1: Feature Extraction (anime dataset)

→ **Loading External data:** Importing external csv dataset into a Pandas DataFrame.

→ **Text Parsing & Feature Extraction:** writing custom python loops & conditions to isolate specific data points (like numbers or dates) hidden within long text strings.

→ **Applying custom Function:** using `.apply()` method to execute custom python functions across an entire DataFrame column.

→ **Data cleaning:** Removing unwanted text characters from extracted data `.str.replace()`

→ **Data Type conversion:** converting string data into mathematical integers using `.astype()`.

→ **Time series calculations:** Utilizing AI-generated logic & and python libraries like **datetime** & **dateutil** to calculate the duration between extracted dates.

Project 2 : Dataset & Analysis (countries dataset)

→ **Data preprocessing & Inspection**: Using methods like `.shape`, `.info()`, `.describe()` to understand dataset dimensions, data types, & find missing values before analysis.

→ **Conditional selection**: Filtering DataFrames based on specific conditions to extract targeted rows.

→ **Finding Extremes**: Locating maximum `.max()` & minimum `.min()` values within a dataset.

→ **Data sorting**: Ranking data using `.sort_values()` & adjusting the order using the ascending parameter.

→ **Frequency counting**: Using `.value_counts()` and `.count()` to determine how many times specific categories or regions appear in the data.

→ **Extracting Ranked Elements** : utilizing `.nlargest()` combined with `.iloc()` to isolate specifically ranked data (ex - the second or third highest values)

→ **Locating Missing Data**: using `.isna()` inside a conditional selection to isolate row with unknown or null values.

→ **String Matching**: Applying custom functions with `.lower()` to scan for specific text patterns (like a specific word) within a column.

→ **Creating subsets**: creating an entirely new, smaller DataFrame by filtering out a specific category from the main dataset for isolated analysis.